CNFSDP

Centre National de Formation en Statistique,
Démographie et Planification
Brazzaville, République du Congo

Introduction au Machine Learning

Chapitre 3 – Classification et méthodes d'ensemble

Plus proches voisins, régression logistique, arbres, forêts et boosting

Rois Céti DIMBAMBA

Ingénieur statisticien économiste · Actuaire · CV en ligne

Licence 3 · Semestre 5 · UE Informatique IV

Année académique 2026-2027 · ?? octobre 2026

Plan et objectifs

1. Le problème de classification
2. Les plus proches voisins
3. La régression logistique
4. Les arbres de décision
5. Bagging et forêts aléatoires
6. Le boosting
7. Régler et comparer
8. En Python
9. TP en salle

Objectifs de la séance

- Connaître trois classifieurs : plus proches voisins, régression logistique, arbre ; les faire tourner à la main.
- Comprendre pourquoi une moyenne d'arbres (bagging, forêt) est plus stable qu'un arbre.
- Comprendre comment le boosting corrige ses erreurs pas à pas.
- Régler et comparer des modèles par validation croisée.

Prérequis : chapitres 1 et 2.

1

Le problème de classification

Classer des individus

Définition 3.1 (Problème de classification binaire). La cible prend deux valeurs, $y \in \{0, 1\}$. On cherche un classifieur f de faible probabilité d'erreur $\mathbb{P}\big(f(X) \neq Y\big)$.

Fil rouge (chiffres fictifs) : 10 enfants ; x_2 est la distance au centre de santé.

enfant	1	2	3	4	5	6	7	8	9	10
x_1 : années d'études de la mère	0	1	3	4	6	6	7	9	10	2
x_2 : distance (km)	9	3	7	2	8	4	9	6	1	5
y : complètement vacciné	0	1	0	1	0	1	1	1	1	0

Rappel du chapitre 1 : avec $\eta(x) = \mathbb{P}(Y = 1 \mid X = x)$, le meilleur classifieur prédit 1 si $\eta(x) > \frac{1}{2}$. Mais η est inconnue.

2

Les plus proches voisins

La méthode des k plus proches voisins

Définition 3.2 (k plus proches voisins, k-NN). Pour un nouvel individu x, on cherche les k individus de l'échantillon les plus proches de x (distance euclidienne). On estime $\eta(x)$ par la proportion de 1 parmi eux :

$$\hat{\eta}(x) = \frac{1}{k} \sum_{i \in V_k(x)} y_i,$$

où $V_k(x)$ est l'ensemble des k voisins. On prédit la classe majoritaire chez les voisins.

- Aucune formule à ajuster : on garde l'échantillon et l'on compare.
- Distance euclidienne : $\|x - x_i\|^2 = (x_1 - x_{i1})^2 + \cdots + (x_d - x_{id})^2$ (chapitre 0).
- Un seul réglage : le nombre de voisins k.

Le calcul sur le fil rouge

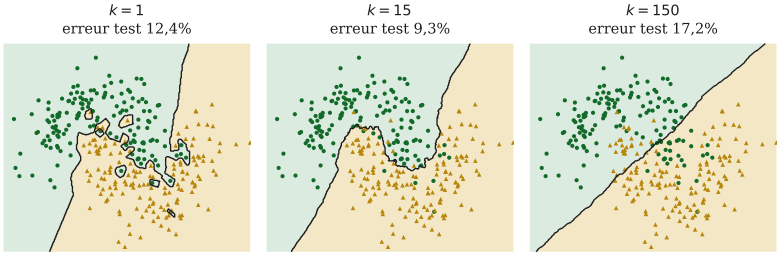
Nouvel enfant : mère avec 5 ans d'études, à 5 km du centre, $x = (5, 5)$.

enfant	6	3	10	4	5	8
position	$(6, 4)$	$(3, 7)$	$(2, 5)$	$(4, 2)$	$(6, 8)$	$(9, 6)$
distance au carré	$1 + 1 = 2$	$4 + 4 = 8$	$9 + 0 = 9$	$1 + 9 = 10$	$1 + 9 = 10$	$16 + 1 = 17$
vacciné	1	0	0	1	0	1

- $k = 1$: le plus proche est vacciné, $\hat{\eta} = 1$: on prédit **vacciné**.
- $k = 3$: voisins 6, 3, 10, $\hat{\eta} = \frac{1}{3}$: on prédit **non vacciné**.
- $k = 5$: $\hat{\eta} = \frac{2}{5}$: **non vacciné**.

Lecture. La prédiction dépend de k. Avec $k = 1$, un seul voisin décide : la règle est très sensible au hasard de l'échantillon.

Choisir k, et standardiser



Deux classes en « croissants » (300 points, données simulées). $k = 1$: frontière tourmentée, sur-apprentissage. $k = 150$: frontière presque droite, sous-apprentissage. On choisit k par validation croisée (ici, $k = 15$ convient).

Standardiser. La distance additionne des écarts dans des unités différentes : un revenu en FCFA écraserait un nombre d'années d'études. On standardise les variables avant le k-NN.

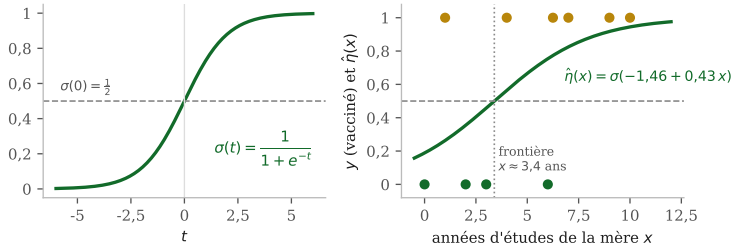
3

La régression logistique

Le modèle logistique

Définition 3.3 (Régression logistique). On modélise la probabilité par

$$\eta_\theta(x) = \sigma(\theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d), \qquad \sigma(t) = \frac{1}{1+e^{-t}} \quad \text{(fonction } \textit{sigmoïde}\text{)}.$$



σ croît de 0 à 1, avec $\sigma(0) = \frac{1}{2}$. À droite : modèle ajusté sur le fil rouge (études seules).

Frontière et lecture des coefficients

Proposition 3.4 (Frontière linéaire)

Le classifieur logistique prédit 1 si et seulement si $\theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d > 0$.

Preuve. σ est strictement croissante et $\sigma(0) = \frac{1}{2}$, donc $\sigma(s) > \frac{1}{2} \iff s > 0$. □

Définition 3.5 (Cote). La *cote* d'un événement de probabilité p est $\frac{p}{1-p}$. Pour le modèle logistique, $\ln \frac{\eta_\theta(x)}{1-\eta_\theta(x)} = \theta_0 + \theta_1 x_1 + \cdots + \theta_d x_d$.

Exemple. [Fil rouge] L'ordinateur donne $\hat{\theta}_0 = -1{,}46$ et $\hat{\theta}_1 = 0{,}43$. Frontière : $x = \frac{1{,}46}{0{,}43} \approx 3{,}4$ ans d'études. Chaque année d'études multiplie la cote de vaccination par $e^{\hat{\theta}_1} \approx 1{,}53$. Pour une mère sans instruction : $\hat{\eta}(0) = \sigma(-1{,}46) \approx 0{,}19$; avec 10 ans : $\hat{\eta}(10) \approx 0{,}94$.

Ajuster le modèle : maximum de vraisemblance

Définition 3.6 (Perte logistique). Pour une probabilité prédite p et une observation $y \in \{0,1\}$: $\ell(p,y) = -\big[y \ln p + (1-y)\ln(1-p)\big]$, soit $-\ln p$ si $y=1$ et $-\ln(1-p)$ si $y=0$.

Proposition 3.7

Maximiser la vraisemblance $\prod_i \eta_\theta(x_i)^{y_i}\big(1-\eta_\theta(x_i)\big)^{1-y_i}$ revient à minimiser la perte logistique moyenne $\frac{1}{n}\sum_i \ell\big(\eta_\theta(x_i), y_i\big)$.

Preuve. Chaque y_i suit une loi de Bernoulli de paramètre $\eta_\theta(x_i)$ (chapitre 0). Le logarithme est croissant et transforme le produit en somme : $\ln(\text{vraisemblance}) = -\sum_i \ell(\eta_\theta(x_i), y_i)$. □

La perte punit fort une confiance mal placée : prédire $p = 0{,}01$ pour un enfant vacciné coûte $-\ln 0{,}01 \approx 4{,}6$.

La perte sur le fil rouge, et le calcul de $\hat{\theta}$

Avec $\hat{\theta} = (-1{,}46;\ 0{,}43)$:

- enfant 4 ($x = 4$, vacciné) : $\hat{\eta} = \sigma(0{,}26) \approx 0{,}56$, perte $-\ln 0{,}56 \approx 0{,}57$;
- enfant 5 ($x = 6$, non vacciné) : $\hat{\eta} = \sigma(1{,}12) \approx 0{,}75$, perte $-\ln 0{,}25 \approx 1{,}40$;
- perte moyenne sur les 10 enfants : $0{,}52$, contre $0{,}67$ pour la constante $\hat{\eta} = 0{,}6$.

Proposition 3.8 (admise)

La perte logistique moyenne est une fonction convexe de θ : elle a un seul minimum, sans formule explicite. Son gradient vaut

$$\frac{\partial}{\partial \theta_j}\Big(\frac{1}{n}\sum_i \ell\Big) = \frac{1}{n}\sum_{i=1}^{n} \big(\eta_\theta(x_i) - y_i\big)\, x_{ij} \qquad (x_{i0} = 1).$$

On le calcule par descente de gradient (chapitre 0). Le gradient est « erreur de probabilité $\times$ variable », comme pour les moindres carrés.

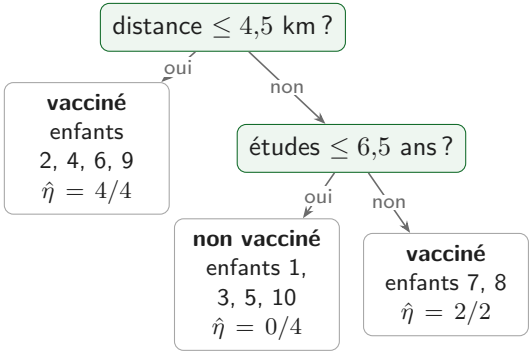
4

Les arbres de décision

L'arbre de classification

Définition 3.9 (Arbre de classification). Un arbre pose des questions « $x_j \leq s$? » et découpe l'espace en zones (feuilles). Dans chaque feuille, $\hat{\eta}$ est la proportion de 1, et l'on prédit la classe majoritaire.

L'arbre ci-contre est appris sur le fil rouge : il classe sans erreur les 10 enfants.



Lecture : près du centre de santé, les enfants sont vaccinés ; loin, cela dépend de l'instruction de la mère.

Choisir les questions : l'indice de Gini

Définition 3.10 (Indice de Gini d'un groupe). Si une proportion p du groupe est dans la classe 1, l'*impureté de Gini* du groupe est $G(p) = 2p(1-p)$. Elle vaut 0 pour un groupe pur ($p = 0$ ou 1) et atteint son maximum $\frac{1}{2}$ pour $p = \frac{1}{2}$.

Une coupure sépare n individus en deux groupes de tailles n_g et n_d. On choisit la coupure qui minimise l'impureté moyenne

$$\frac{n_g}{n}\, G(p_g) + \frac{n_d}{n}\, G(p_d).$$

Remarque. Avec K classes, $G = 1 - \sum_k p_k^2$. On peut aussi utiliser l'entropie $-\sum_k p_k \ln p_k$; les deux donnent en général des arbres proches.

Le calcul sur le fil rouge

Avant toute coupure : $p = \frac{6}{10}$, $G = 2 \times 0{,}6 \times 0{,}4 = 0{,}48$.

	groupe « oui »			groupe « non »		
coupure	effectif	p	G	effectif	p	G
distance $\leq 4{,}5$	4	1	0	6	$\frac{1}{3}$	$\frac{4}{9}$
études $\leq 3{,}5$	4	$\frac{1}{4}$	0,375	6	$\frac{5}{6}$	$\frac{5}{18}$
distance $\leq 6{,}5$	6	$\frac{5}{6}$	$\frac{5}{18}$	4	$\frac{1}{4}$	0,375

Impureté moyenne : distance $\leq 4{,}5$: $0{,}4 \times 0 + 0{,}6 \times \frac{4}{9} = \mathbf{0{,}267}$; les deux autres : $0{,}4 \times 0{,}375 + 0{,}6 \times \frac{5}{18} = 0{,}317$. On essaie toutes les coupures possibles des deux variables : la meilleure est « distance $\leq 4{,}5$ ». On recommence dans le groupe impur (« non ») : « études $\leq 6{,}5$ » le sépare parfaitement.

Un arbre poussé jusqu'au bout sur-apprend (chapitre 1) : on limite sa profondeur ou la taille des feuilles, réglées par validation croisée.

Forces et faiblesses d'un arbre

Forces

- Lisible : une suite de questions simples, que l'on peut expliquer.
- Pas besoin de standardiser ; variables qualitatives faciles.
- Capte des effets non linéaires et des interactions (la distance compte, puis l'instruction).

Faiblesse majeure : l'instabilité

- Changer quelques individus peut changer la première question, donc tout l'arbre.
- Un arbre profond a une forte *variance* (chapitre 1, TP1).

À retenir

Idée des méthodes d'ensemble : combiner beaucoup d'arbres. Le **bagging** fait la moyenne d'arbres instables pour réduire la variance ; le **boosting** additionne de petits arbres pour réduire le biais.

5

Bagging et forêts aléatoires

Bootstrap et bagging

Définition 3.11 (Échantillon bootstrap, bagging). Un *échantillon bootstrap* est obtenu en tirant n individus *avec remise* parmi les n de l'échantillon : certains apparaissent plusieurs fois, d'autres pas du tout. Le *bagging* (*bootstrap aggregating*) apprend un arbre $\hat{f}_b$ sur chacun de B échantillons bootstrap, puis agrège :

$$\hat{\eta}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{\eta}_b(x) \qquad \text{(ou vote majoritaire des } B \text{ arbres).}$$

Exemple. [Fil rouge] Un tirage avec remise de 10 enfants parmi 10 : $(3, 7, 7, 1, 9, 4, 4, 4, 10, 6)$. Les enfants 2, 5 et 8 ne sont pas tirés ; l'arbre appris sur ce tirage sera différent de celui du cours.

Pourquoi la moyenne stabilise

Proposition 3.12 (Variance d'une moyenne)

Si $Z_1, \dots, Z_B$ sont indépendantes, de même variance σ^2, alors $\operatorname{Var}\left(\frac{1}{B}\sum_{b=1}^{B} Z_b\right) = \frac{\sigma^2}{B}$.

Preuve. Par indépendance, la variance d'une somme est la somme des variances (chapitre 0) : $\operatorname{Var}\left(\sum_b Z_b\right) = B\sigma^2$. Et $\operatorname{Var}(cZ) = c^2 \operatorname{Var}(Z)$ avec $c = \frac{1}{B}$. □

Ici $Z_b = \hat{\eta}_b(x)$, la prédiction du b-ième arbre en un point x. Mais les arbres sont appris sur des tirages du *même* échantillon : ils sont corrélés. Si leur corrélation vaut ρ (admis) :

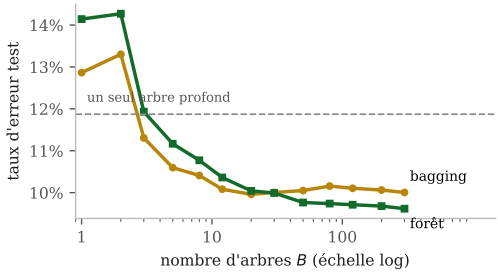
$$\operatorname{Var}\left(\frac{1}{B}\sum_{b} \hat{\eta}_b(x)\right) = \rho\,\sigma^2 + \frac{(1-\rho)\,\sigma^2}{B} \xrightarrow[B\to\infty]{} \rho\,\sigma^2.$$

Exemple : $\sigma^2 = 1$, $B = 100$: variance $0{,}505$ si $\rho = 0{,}5$, mais $0{,}109$ si $\rho = 0{,}1$. Pour aller plus loin, il faut *décorréler* les arbres.

La forêt aléatoire

Définition 3.13 (Forêt aléatoire). C'est un bagging d'arbres profonds où, *à chaque coupure*, l'arbre ne choisit qu'entre m variables tirées au hasard parmi les d (souvent $m \approx \sqrt{d}$).

- Si une variable domine, tous les arbres du bagging coupent d'abord sur elle : ils se ressemblent (ρ élevé).
- En cachant des variables au hasard, on force des arbres variés : ρ baisse, la moyenne est plus stable.
- Plus d'arbres ne fait pas sur-apprendre : l'erreur se stabilise.



L'erreur out-of-bag

Proposition 3.14

Dans un échantillon bootstrap de taille n, un individu donné n'est pas tiré avec probabilité $\left(1 - \frac{1}{n}\right)^n$, qui tend vers $e^{-1} \approx 0{,}37$ quand n grandit (limite admise).

Preuve. Chacun des n tirages indépendants manque l'individu avec probabilité $1 - \frac{1}{n}$. □

Pour $n = 10$: $0{,}9^{10} \approx 0{,}35$. Environ un tiers des individus est *hors du sac* (*out-of-bag*) pour chaque arbre.

Définition 3.15 (Erreur out-of-bag). Pour chaque individu i, on le classe en ne faisant voter que les arbres qui ne l'ont pas vu, puis on calcule le taux d'erreur sur tous les i.

C'est une validation « gratuite » : elle estime l'erreur test sans mettre de données de côté.

Quelles variables comptent ?

Définition 3.16 (Importance des variables).

- **Par impureté** : on additionne, sur tous les arbres, la baisse d'impureté de Gini obtenue aux coupures sur la variable x_j.
- **Par permutation** : on mélange au hasard la colonne x_j dans les données test et l'on mesure la hausse de l'erreur. Si l'erreur ne bouge pas, le modèle n'utilise pas x_j.

Précautions.

- Une variable importante pour le modèle n'est pas une cause : c'est une variable *utile pour prédire*.
- Deux variables très corrélées se partagent l'importance.
- L'importance par permutation est plus fiable ; nous y reviendrons au chapitre 5.

6

Le boosting

AdaBoost : des votes pondérés

Ici les classes sont codées $y \in \{-1, +1\}$ et l'on combine des arbres à une seule coupure, $h(x) \in \{-1, +1\}$.

Définition 3.17 (AdaBoost). Poids initiaux $w_i = \frac{1}{n}$. Pour $m = 1, \dots, M$:

1. choisir la coupure h_m d'erreur pondérée $\varepsilon_m = \sum_{i:\, h_m(x_i) \neq y_i} w_i$ minimale ;
2. lui donner le poids de vote $\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$;
3. multiplier w_i par e^{α_m} si i est mal classé, par $e^{-\alpha_m}$ s'il est bien classé, puis renormaliser ($\sum_i w_i = 1$).

Classifieur final : $\hat{f}(x) = \operatorname{signe}\big(\alpha_1 h_1(x) + \cdots + \alpha_M h_M(x)\big)$.

$\alpha_m > 0$ dès que $\varepsilon_m < \frac{1}{2}$; plus la coupure est bonne, plus son vote pèse.

Pourquoi le boosting change de point de vue

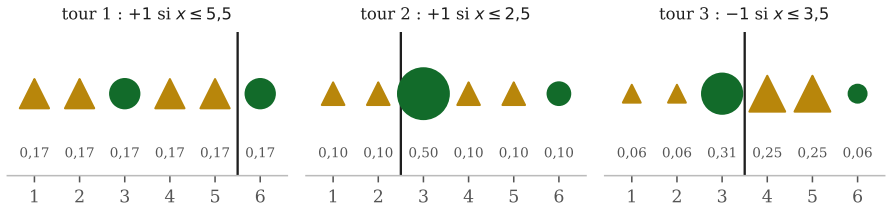
Proposition 3.18

Après la mise à jour des poids du tour m, les individus mal classés par h_m pèsent exactement la moitié du poids total.

Preuve. Avant renormalisation, la masse des mal classés devient $\varepsilon_m e^{\alpha_m} = \varepsilon_m \sqrt{\frac{1-\varepsilon_m}{\varepsilon_m}} = \sqrt{\varepsilon_m(1-\varepsilon_m)}$, celle des bien classés $(1-\varepsilon_m)e^{-\alpha_m} = \sqrt{\varepsilon_m(1-\varepsilon_m)}$: les deux masses sont égales. □

Lecture. Pour les nouveaux poids, h_m ne fait pas mieux que le hasard (erreur $\frac{1}{2}$). Le tour suivant est donc obligé de trouver une *autre* coupure, qui s'occupe des individus difficiles.

AdaBoost à la main sur six points



Triangles : $y = +1$; ronds : $y = -1$. La taille et le nombre sous chaque point donnent son poids au début du tour.

tour	coupure h_m	mal classés	ε_m	$\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$
1	$+1$ si $x \leq 5{,}5$	point 3	$\frac{1}{6}$	$\frac{1}{2} \ln 5 \approx 0{,}80$
2	$+1$ si $x \leq 2{,}5$	points 4, 5	$0{,}1 + 0{,}1 = 0{,}2$	$\frac{1}{2} \ln 4 \approx 0{,}69$
3	-1 si $x \leq 3{,}5$	points 1, 2, 6	$0{,}0625 \times 3 = 0{,}1875$	$\approx 0{,}73$

Le vote final

$F(x) = 0{,}80\,h_1(x) + 0{,}69\,h_2(x) + 0{,}73\,h_3(x)$ et $\hat{f}(x) = \operatorname{signe} F(x)$.

x	y	h_1	h_2	h_3	$F(x)$	prédiction
1, 2	$+1$	$+1$	$+1$	-1	$0{,}80 + 0{,}69 - 0{,}73 = 0{,}76$	$+1$
3	-1	$+1$	-1	-1	$0{,}80 - 0{,}69 - 0{,}73 = -0{,}62$	-1
4, 5	$+1$	$+1$	-1	$+1$	$0{,}80 - 0{,}69 + 0{,}73 = 0{,}84$	$+1$
6	-1	-1	-1	$+1$	$-0{,}80 - 0{,}69 + 0{,}73 = -0{,}76$	-1

À retenir

Chaque coupure seule se trompe sur au moins un point ; leur vote pondéré classe les six points sans erreur. Des classifieurs *faibles* deviennent ensemble un classifieur *fort*.

scikit-learn (`AdaBoostClassifier`) affiche des poids $2\alpha_m$: même vote, même classement.

Gradient boosting, et bilan des méthodes d'ensemble

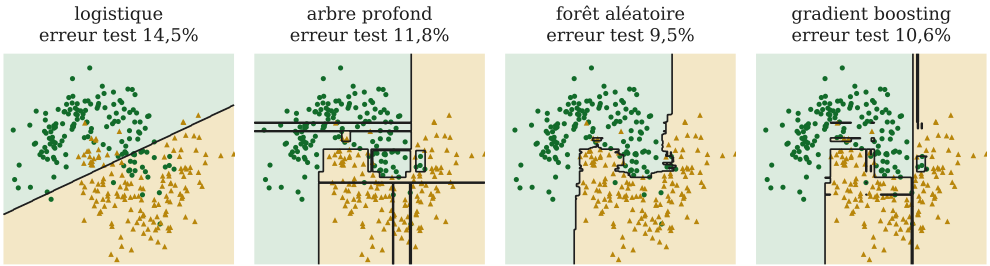
Gradient boosting en classification : même principe qu'au chapitre 2. On part d'un score constant, et chaque petit arbre est ajusté sur les écarts $y_i - \hat{\eta}(x_i)$ entre la classe observée et la probabilité prédite ; on avance d'un pas ν (taux d'apprentissage).

	Bagging, forêt	**Boosting (AdaBoost, gradient)**
Arbres	profonds, indépendants	petits, construits l'un après l'autre
Agrégation	moyenne ou vote simple	somme pondérée
Réduit	la variance	le biais
Sur-apprend si	rarement	trop d'itérations ou ν trop grand
Réglages	B, m, taille des feuilles	M, ν, profondeur

7

Régler et comparer

Des frontières très différentes



Mêmes données que pour le k-NN. La logistique trace une droite : trop rigide pour des croissants. L'arbre profond découpe en rectangles et colle au bruit. La forêt et le boosting combinent des rectangles en une frontière souple et stable.

Régler les hyperparamètres par validation croisée

Définition 3.19 (Hyperparamètre, recherche sur grille). Un *hyperparamètre* est un réglage choisi avant l'apprentissage (k, profondeur, nombre d'arbres…). La *recherche sur grille* essaie toutes les combinaisons d'une liste, estime l'erreur de chacune par validation croisée et garde la meilleure.

Modèle	Hyperparamètres principaux
k-NN	nombre de voisins k
logistique	pénalité (Ridge ou Lasso, chapitre 2)
arbre	profondeur maximale, taille minimale des feuilles
forêt	nombre d'arbres, variables tirées m, taille des feuilles
boosting	nombre d'arbres M, taux ν, profondeur

Le jeu test reste à l'écart de la grille : on l'utilise une seule fois, à la fin (chapitre 1).

8

En Python

k-NN, logistique et arbre, réglés par grille

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

X, y = load_breast_cancer(return_X_y=True)    # 569 tumeurs, 30 mesures ; y = 1 : bénigne
Xa, Xt, ya, yt = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
knn = GridSearchCV(make_pipeline(StandardScaler(), KNeighborsClassifier()),
                   {"kneighborsclassifier__n_neighbors": [1, 3, 5, 9, 15, 25]}, cv=5)
modeles = {"k-NN": knn,
           "logistique": make_pipeline(StandardScaler(), LogisticRegression()),
           "arbre": GridSearchCV(DecisionTreeClassifier(random_state=0),
                                 {"max_depth": [1, 2, 3, 4, 6, 8, None]}, cv=5)}
for nom, f in modeles.items():
    print(nom, round(f.fit(Xa, ya).score(Xt, yt), 3))    # exactitude test
# k-NN 0.947 | logistique 0.959 | arbre 0.906   (k = 5 et profondeur 2 retenus)
```

Bagging, forêt et boosting

```python
import numpy as np
from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
                              AdaBoostClassifier, GradientBoostingClassifier)
souche = DecisionTreeClassifier(max_depth=1)          # arbre à une coupure
noms = load_breast_cancer().feature_names
modeles = {"arbre seul": DecisionTreeClassifier(random_state=0),
           "bagging": BaggingClassifier(DecisionTreeClassifier(),
                                        n_estimators=300, random_state=0),
           "forêt": RandomForestClassifier(n_estimators=300, oob_score=True, random_state=0),
           "AdaBoost": AdaBoostClassifier(souche, n_estimators=300, random_state=0),
           "boosting": GradientBoostingClassifier(random_state=0)}
for nom, f in modeles.items():
    print(nom, round(f.fit(Xa, ya).score(Xt, yt), 3))
foret = modeles["forêt"]
print(round(foret.oob_score_, 3), noms[np.argsort(foret.feature_importances_)[::-1][:3]])
# arbre seul 0.906 | bagging 0.93 | forêt 0.953 | AdaBoost 0.971 | boosting 0.936
# 0.967 (OOB) ['worst perimeter' 'worst radius' 'worst concave points']
```

Sur 171 tumeurs test, l'IC à $95\,\%$ fait environ ± 3 points : seul l'arbre seul est nettement derrière.

9

TP en salle

TP3 – Classifieurs et méthodes d'ensemble

TP3 – à faire en salle — 45 min · `notebooks/TP3.ipynb` (Colab)

1. **k-NN à la main** : sur le fil rouge, retrouver les prédictions du cours pour $x = (5, 5)$ et $k = 1, 3, 5$; vérifier avec `KNeighborsClassifier`.
2. **Choix de k** : sur les croissants, tracer l'erreur de validation croisée pour $k = 1, \ldots, 50$.
3. **Logistique** : retrouver $\hat{\theta} = (-1{,}46;\ 0{,}43)$ sur le fil rouge, la cote $e^{\hat{\theta}_1}$, et vérifier que le gradient est nul.
4. **Gini** : écrire une fonction qui calcule l'impureté d'une coupure ; retrouver la meilleure coupure du cours.
5. **Bagging en NumPy** : B arbres sur des tirages bootstrap, vote majoritaire ; comparer à `BaggingClassifier` et à un arbre seul.
6. **Forêt** : erreur out-of-bag contre erreur test ; trois variables les plus importantes.
7. **AdaBoost à la main** : refaire les trois tours du cours ; comparer à `AdaBoostClassifier`.

Résumé

- Classer, c'est estimer $\eta(x) = \mathbb{P}(Y = 1 \mid X = x)$ puis seuiller (principe plug-in).
- k-NN : proportion de 1 chez les k voisins ; k petit sur-apprend ; standardiser.
- Logistique : $\hat{\eta} = \sigma(\theta_0 + \theta^\top x)$, frontière linéaire, coefficients lus en cotes ; ajustée par maximum de vraisemblance, c'est-à-dire en minimisant la perte logistique.
- Arbre : questions successives choisies par l'indice de Gini ; lisible mais instable.
- Bagging et forêt : la moyenne de B arbres divise la variance (par B s'ils étaient indépendants) ; la forêt décorrèle les arbres ; erreur out-of-bag gratuite.
- Boosting : de petits arbres qui se concentrent sur les erreurs ; vote pondéré (AdaBoost) ou ajustement des écarts (gradient boosting).
- Hyperparamètres réglés par validation croisée ; comparaison finale sur le test, avec son intervalle de confiance.